Gestão de Rede Centralizada e SDN

Infraestruturas de Computação na Nuvem, Aula 06

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

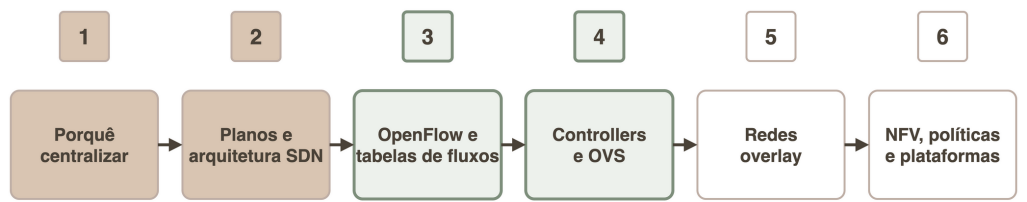
Agenda (1/2)

- Limitações da gestão de rede tradicional em datacenters.
- Separação entre plano de controlo e plano de dados.
- Arquitetura de Software-Defined Networking (SDN).
- OpenFlow e tabelas de fluxos.
- Controllers SDN e o modelo reativo vs proativo.

Agenda (2/2)

- Open vSwitch e redes de containers/Kubernetes.
- Redes overlay: VLAN, VXLAN e outros encapsulamentos.
- Network Functions Virtualization (NFV).
- Microssegmentação e políticas de rede.
- SDN no Proxmox e no Kubernetes.
- Vantagens e desafios da centralização.

Percurso da Aula

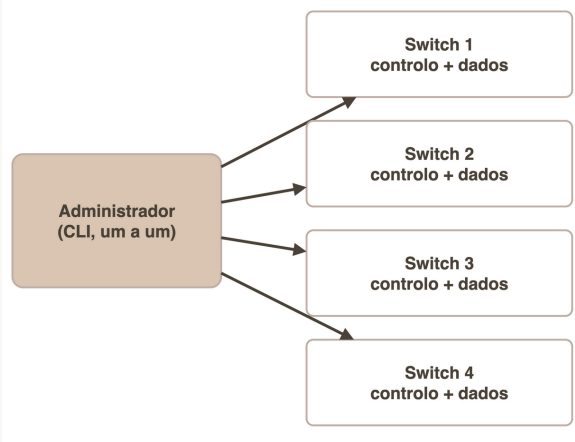


Da decisão distribuída por equipamento à rede tratada como software

Gestão de Rede Tradicional

- Cada equipamento de rede (switch, router) tem o seu próprio plano de controlo, configurado individualmente.
- Configuração distribuída, dispositivo a dispositivo, tipicamente via CLI proprietária.
- Em datacenters com centenas ou milhares de switches, este modelo não escala: lento, sujeito a erro humano e difícil de auditar.
- Motivação central da SDN: mover a inteligência de decisão para fora dos equipamentos individuais.

Os Limites do Modelo Tradicional



Lento
centenas de equipamentos, um a um

Sujeito a erro
configurações divergentes

Difícil de auditar
sem visão global da rede

Plano de Controlo vs Plano de Dados

- **Plano de dados (data plane)**: encaminha pacotes com base em regras já instaladas – rápido, executado em hardware/software do switch.
- **Plano de controlo (control plane)**: decide *quais* são essas regras – routing, políticas, topologia.
- Nas redes tradicionais, os dois planos coexistem em cada equipamento.
- Na SDN, o plano de controlo é retirado dos equipamentos e centralizado num **controller** externo.

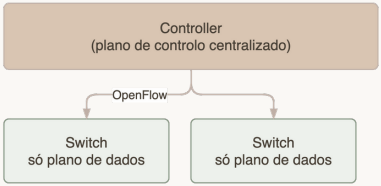
Onde Vive a Decisão

Rede tradicional



Cada equipamento decide sozinho.
Configuração dispositivo a dispositivo.

SDN

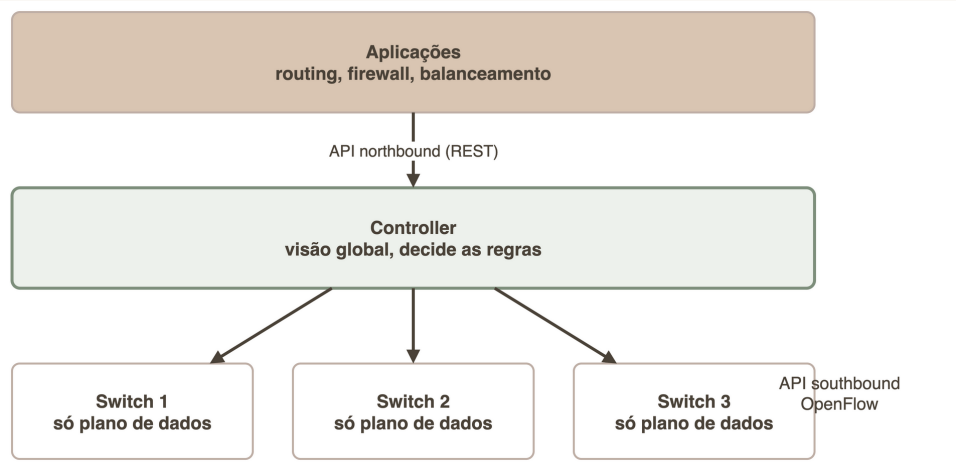


Visão global, políticas consistentes.
O controller torna-se crítico.

Arquitetura SDN

- **Camada de infraestrutura**: switches simples, apenas com plano de dados (encaminham segundo regras recebidas).
- **Controller (plano de controlo)**: elemento centralizado com visão global da rede, decide as regras.
- **Camada de aplicações**: software que define políticas (routing, firewall, balanceamento) através do controller.
- **API southbound**: comunicação controller $\leftrightarrow$ switches (ex. OpenFlow).
- **API northbound**: comunicação controller $\leftrightarrow$ aplicações.

As Camadas da Arquitetura SDN



OpenFlow e Tabelas de Fluxos

- OpenFlow é o protocolo southbound mais usado para comunicação entre controller e switches.
- Cada switch mantém uma **tabela de fluxos**, com entradas do tipo *match* – *action*.
- **Match**: campos do pacote (IP origem/destino, porta, protocolo, ...).
- **Action**: encaminhar, descartar, reencaminhar para o controller, modificar cabeçalho.
- Quando não há regra aplicável, o pacote é enviado ao controller, que decide e instala uma nova regra.

Anatomia de uma Entrada de Fluxo

- Cada entrada da tabela de fluxos tem, além do *match* e das *actions*, três elementos importantes:
- **Prioridade**: quando várias entradas correspondem ao mesmo pacote, aplica-se a de maior prioridade.
- **Contadores**: número de pacotes e bytes que corresponderam à regra – base para monitorização e estatísticas.
- **Timeouts**: *idle timeout* (remove a entrada após um período sem uso) e *hard timeout* (remove-a ao fim de um tempo fixo).
- As tabelas têm capacidade limitada, pelo que a gestão de timeouts é essencial em redes com muitos fluxos.

Uma Tabela de Fluxos Concreta

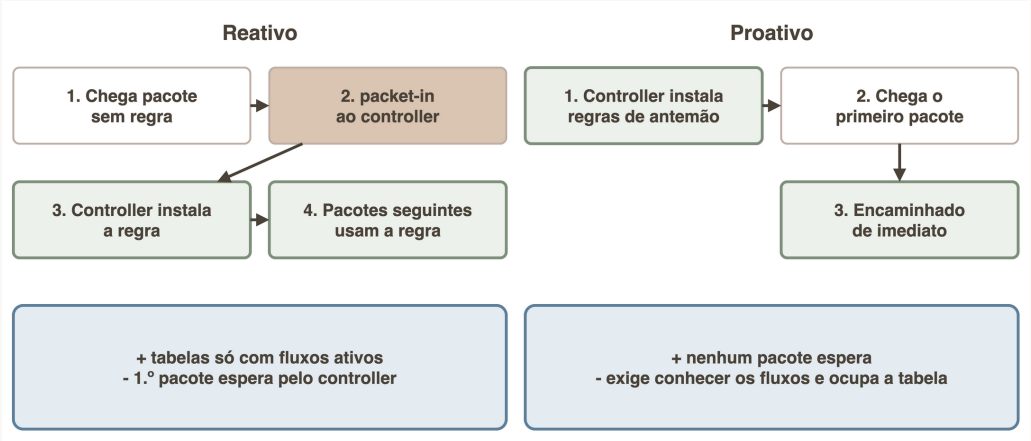
Prioridade	Match	Ação	Contadores	Timeout
100	ip, src 10.0.0.1, dst 10.0.0.3	drop	12 pkt	hard 60 s
10	eth_dst 00:..:02, in_port 1	output:2	840 pkt	idle 30 s
1	eth_dst 00:..:01, in_port 2	output:1	835 pkt	idle 30 s
0	qualquer pacote (table-miss)	enviar ao controller	7 pkt	sem

Vence a regra de maior prioridade que corresponda ao pacote

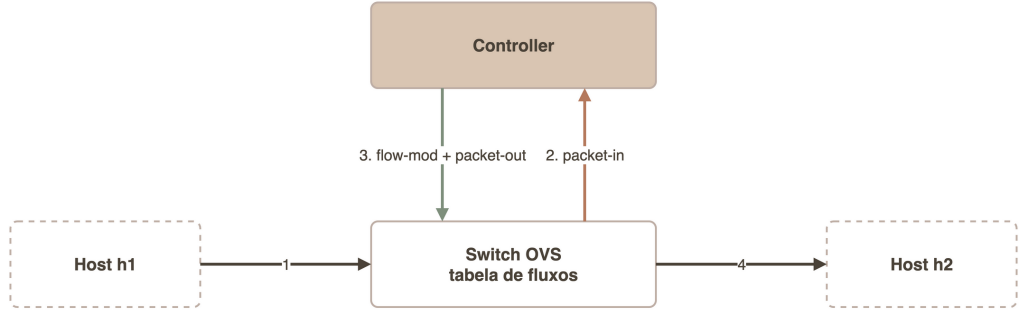
Modelo Reativo vs Proativo

- **Reativo**: as regras só são instaladas quando chega o primeiro pacote de um fluxo desconhecido (*packet-in* para o controller).
- Vantagem: as tabelas contêm apenas fluxos ativos. Desvantagem: o primeiro pacote sofre atraso e o controller pode tornar-se um estrangulamento.
- **Proativo**: as regras são instaladas antecipadamente, antes de haver tráfego.
- Vantagem: nenhum pacote espera pelo controller. Desvantagem: exige conhecer previamente os fluxos e ocupa espaço nas tabelas.
- Implementações reais combinam frequentemente as duas abordagens.

Reativo e Proativo em Comparação



O Percurso de um Pacote sem Regra



1. Chega um pacote de h1
2. Sem regra aplicável: packet-in
3. O controller instala a regra

4. Os pacotes seguintes seguem a regra, sem passar pelo controller

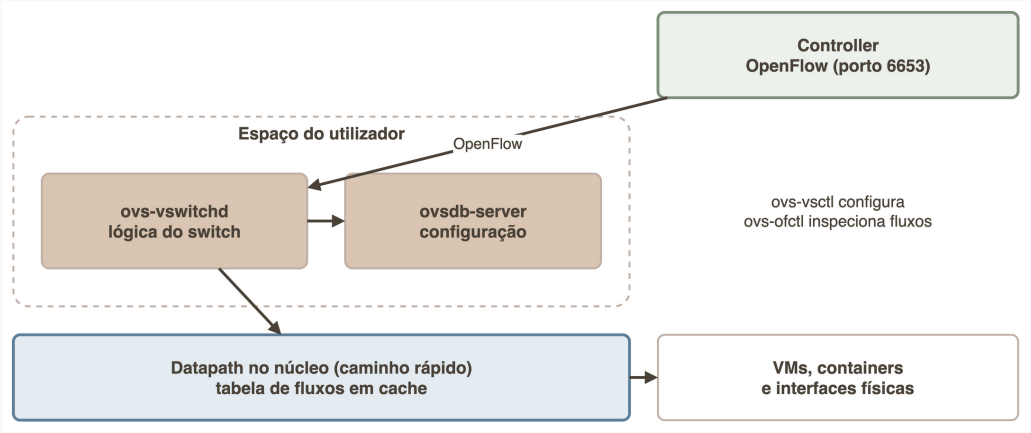
Controllers SDN

- **Ryu**: controller em Python, simples e didático; é o que usaremos no laboratório.
- **OpenDaylight** e **ONOS**: plataformas de grande escala, em Java, orientadas a operadores de telecomunicações.
- **Faucet**: controller leve, focado em ambientes de produção com configuração declarativa.
- A **API northbound** (tipicamente REST) permite que aplicações externas definam políticas sem falar OpenFlow diretamente.

Open vSwitch (OVS)

- Open vSwitch é um switch virtual, programável via OpenFlow, amplamente usado em hipervisores e datacenters.
- Substitui a bridge de rede tradicional em hosts que correm máquinas virtuais ou containers.
- Relembrando a Aula 05: as redes que o Kubernetes usa para ligar Pods entre nós assentam frequentemente em switches programáveis deste tipo.
- Permite aplicar políticas de rede (isolamento, QoS, monitorização) de forma centralizada e programática.

Open vSwitch por Dentro



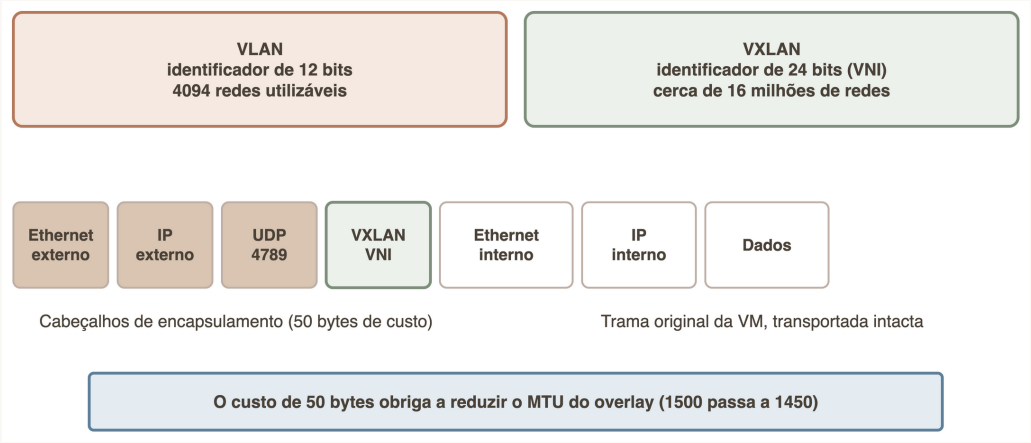
Redes Overlay: VXLAN

- Datacenters de cloud precisam de muitas redes virtuais isoladas sobre a mesma infraestrutura física.
- VLANs tradicionais limitam-se a 4094 identificadores – insuficiente à escala de cloud.
- **VXLAN** encapsula tráfego de camada 2 dentro de pacotes UDP (camada 3), criando redes overlay independentes da topologia física.
- Permite milhões de segmentos de rede lógicos e mobilidade de VMs/containers entre servidores físicos sem reconfigurar a rede física.

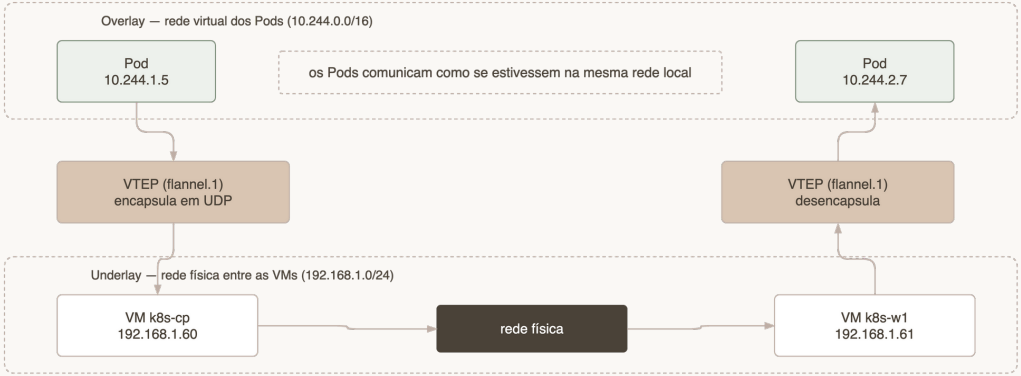
Encapsulamentos de Overlay

- **VXLAN**: encapsula tramas de camada 2 em UDP; identificador de 24 bits (VNI), permitindo cerca de 16 milhões de segmentos.
- **GRE** e **Geneve**: alternativas de encapsulamento; o Geneve foi desenhado para ser extensível através de opções variáveis.
- Terminologia VXLAN: o **VTEP** (VXLAN Tunnel Endpoint) é o ponto onde o encapsulamento é aplicado e removido.
- Custo: o cabeçalho adicional reduz o espaço útil de cada pacote, obrigando a ajustar o MTU da rede subjacente.
- Distinção útil: a rede física subjacente designa-se **underlay**; a rede virtual construída sobre ela, **overlay**.

VLAN, VXLAN e o Quadro Encapsulado



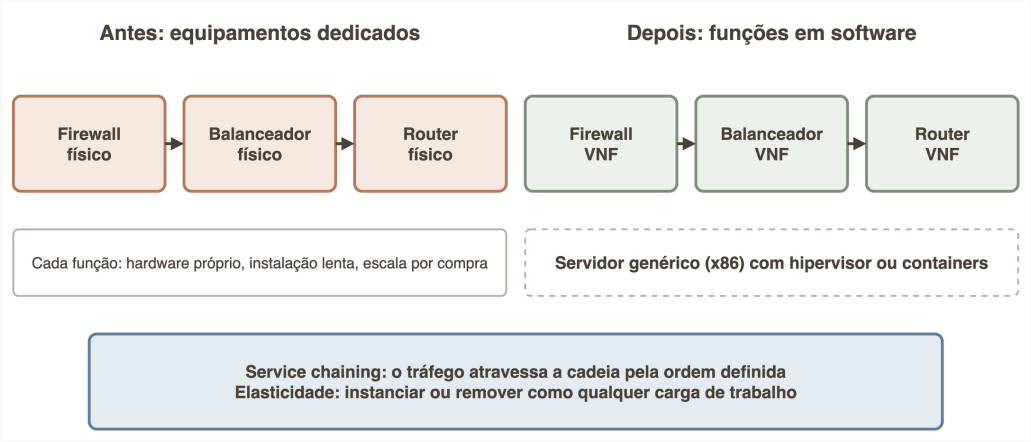
Overlay sobre Underlay



Network Functions Virtualization (NFV)

- Conceito complementar da SDN: substituir equipamento de rede dedicado por software em servidores genéricos.
- Exemplos de funções virtualizadas: firewalls, balanceadores de carga, routers, gateways de VPN.
- **SDN vs NFV**: a SDN trata de *como* o tráfego é encaminhado; a NFV trata de *onde* correm as funções de rede.
- **Service chaining**: encadear várias funções virtualizadas pelas quais o tráfego deve passar sequencialmente.
- Vantagem central: elasticidade – instanciar ou remover funções de rede tão facilmente como qualquer outra carga de trabalho.

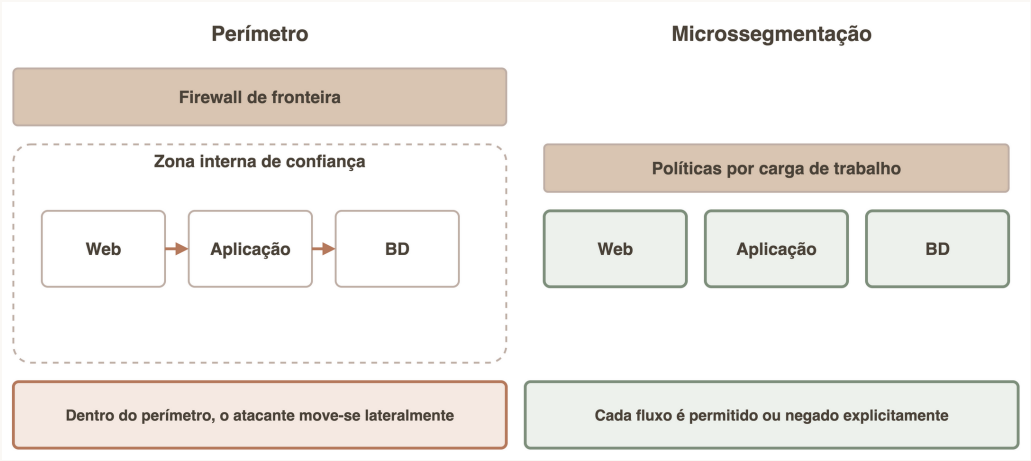
Do Equipamento Dedicado à Função Virtual



Microssegmentação e Políticas de Rede

- Modelo tradicional: um perímetro fortificado (firewall na fronteira) e confiança implícita no interior da rede.
- Problema: uma vez ultrapassado o perímetro, um atacante move-se lateralmente com pouca resistência.
- **Microssegmentação**: aplicar políticas de segurança ao nível de cada carga de trabalho, e não apenas na fronteira.
- A programabilidade da SDN torna isto viável em escala: as políticas são definidas centralmente e aplicadas em cada switch virtual.
- No Kubernetes, o recurso **NetworkPolicy** concretiza esta ideia, restringindo que Pods podem comunicar entre si.

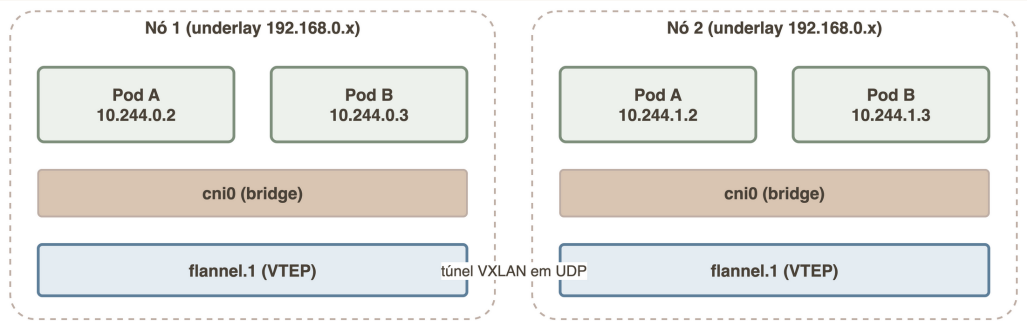
Perímetro e Microssegmentação



SDN no Proxmox e no Kubernetes

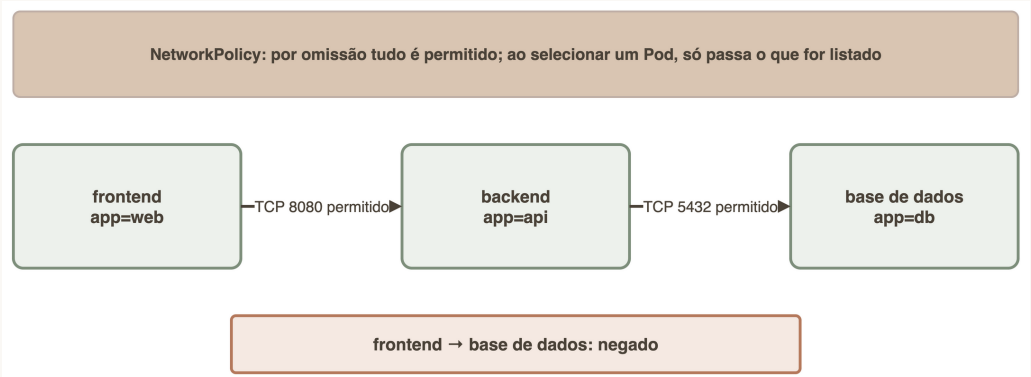
- O Proxmox VE liga as VMs através de **bridges Linux** (ex. `vmbr0`) – foi o que usámos nos guiões anteriores.
- Dispõe também de um módulo **SDN** próprio, que permite definir *zonas* (incluindo VXLAN) e *VNets* sobre a infraestrutura física.
- No Kubernetes, o plugin CNI instalado na Aula 05 desempenha exatamente o papel de camada de rede programável.
- O Flannel, por omissão, cria uma rede overlay entre os nós – tipicamente com encapsulamento VXLAN.
- Ou seja: o cluster montado na aula anterior já usava, sem o dizermos explicitamente, os conceitos desta aula.

A Rede de Pods do Kubernetes



O Pod A do nó 1 fala com o Pod do nó 2 sem saber que existe um túnel

NetworkPolicy em Ação

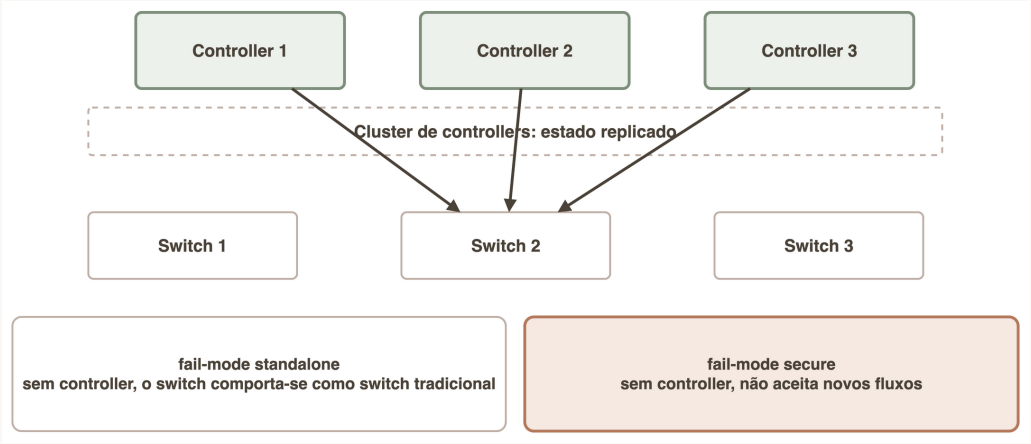


Exige um plugin CNI que aplique políticas (Calico, Cilium); o Flannel sozinho não as aplica

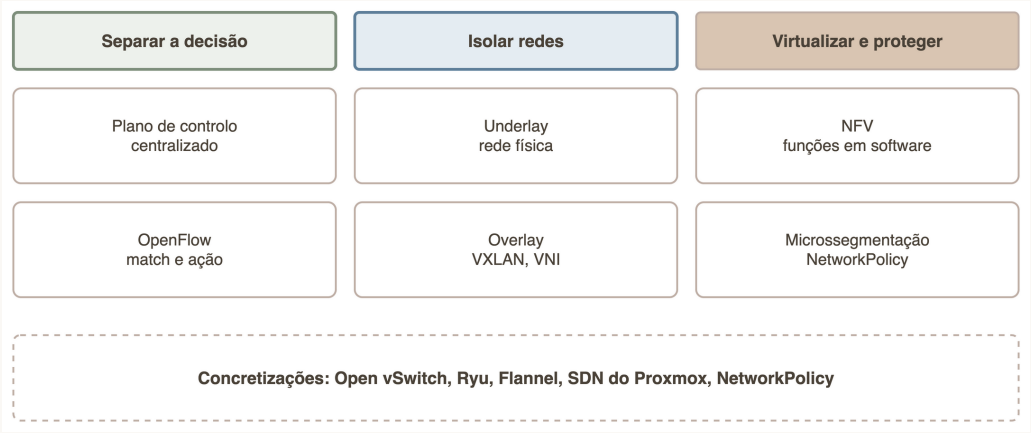
Vantagens e Desafios da Centralização

- **Vantagens**: visão global da rede, políticas consistentes, automação, programabilidade (rede tratada como software).
- **Desafio**: o controller é um ponto crítico – se falhar, pode comprometer toda a rede.
- Mitigação: controllers em cluster (alta disponibilidade), replicando o padrão já visto para outros serviços na Aula 08/09.
- **Segurança**: o canal controller-switch tem de ser protegido, pois concentra o poder de decisão de toda a rede.

O Controller como Ponto Crítico



Mapa da Aula


Separar a decisão
Isolar redes
Virtualizar e proteger
Plano de controlo
centralizado
Underlay
rede física
NFV
funções em software
OpenFlow
match e ação
Overlay
VXLAN, VNI
Microssegmentação
NetworkPolicy
Concretizações: Open vSwitch, Ryu, Flannel, SDN do Proxmox, NetworkPolicy

Síntese da Aula (1/2)

- SDN separa o plano de controlo (centralizado) do plano de dados (distribuído pelos switches).
- OpenFlow é o protocolo southbound típico, baseado em tabelas de fluxos match-action.
- Open vSwitch materializa este modelo em hipervisores e infraestrutura de containers.

Síntese da Aula (2/2)

- VXLAN e outros encapsulamentos criam redes overlay escaláveis sobre um underlay físico, essenciais em datacenters multi-tenant.
- NFV virtualiza as próprias funções de rede; a microssegmentação aplica políticas por carga de trabalho, não só no perímetro.
- O Flannel do cluster da Aula 05 e as bridges/SDN do Proxmox são instâncias concretas destes conceitos.
- A centralização traz consistência e automação, mas exige atenção à disponibilidade e segurança do controller.

Obrigada.

Questões e comentários